

TREINAMENTO - PROVA A1

FICHEIRO 1: AVALIAÇÃO A1 PRÁTICA DE TREINO (FASTBITE / ENTREGA INTELIGENTE)

QUESTÃO 1: Classificação do Problema de Roteamento

a) No âmbito da teoria da complexidade computacional, a **Classe P** engloba todos os problemas de decisão que podem ser resolvidos por um algoritmo determinístico em tempo polinomial, apresentando um crescimento computacional controlável. A **Classe NP** (*Nondeterministic Polynomial time*) contém os problemas para os quais uma solução candidata proposta pode ser verificada quanto à sua validade em tempo polinomial por um algoritmo determinístico, embora encontrar essa solução do zero possa não ser eficiente. Por sua vez, a classe **NP-Completo** agrupa os problemas mais difíceis dentro de NP; um problema pertence a esta classe se estiver em NP e se qualquer outro problema de NP puder ser reduzido a ele em tempo polinomial.

b) O problema de roteamento e atribuição combinados da FastBite classifica-se formalmente como **NP-Completo** (ou NP-Difícil na sua formulação matemática de otimização contínua). Esta classificação justifica-se pelo facto de o Problema de Roteamento de Veículos (VRP) constituir uma generalização direta do Problema do Caixeiro Viajante (TSP), que pertence comprovadamente à classe NP-Completo.

c) A redução intuitiva do cenário da FastBite ao TSP realiza-se mapeando cada ponto de recolha (restaurantes) e cada ponto de entrega (clientes) como um vértice ("cidade") no grafo do TSP. As distâncias Manhattan calculadas entre todas as coordenadas geográficas estabelecem as arestas ponderadas com os respetivos pesos numéricos. A sequência ideal que minimiza o tempo total de deslocamento do estafeta equivale, portanto, à determinação de um circuito hamiltoniano de menor custo dentro do TSP, sendo esta transformação geométrica executada em tempo polinomial $O(n^2)$ face ao tamanho da entrada.

d) A existência de uma redução polinomial ao TSP implica que o problema da FastBite é pelo menos tão difícil quanto o próprio TSP. Se fosse descoberto um algoritmo capaz de resolver a alocação da FastBite em tempo polinomial, esse mesmo algoritmo resolveria qualquer instância genérica do TSP através da redução matemática construída. Como o TSP é NP-Completo, isto resultaria na prova de que $P = NP$, colidindo com a forte conjectura científica de que $P \neq NP$.

QUESTÃO 2: Crescimento Combinatório e Inviabilidade da Força Bruta

a) Para distribuir 8 pedidos por 3 estafetas (com capacidade máxima de 3 pedidos cada), considerando uma partição ordenada específica do tipo 3-3-2 (três pedidos para o primeiro estafeta, três para o segundo e dois para o terceiro), o raciocínio combinatório passo a passo é o seguinte:

Escolha e ordenação interna dos pedidos do estafeta 1: $C(8,3) \times 3!$
 $[cite_start]= 56 \times 6 = 336$

Escolha e ordenação interna dos pedidos do estafeta 2: $C(5,3) \times 3!$
 $[cite_start]= 10 \times 6 = 60$

Escolha e ordenação interna dos pedidos do estafeta 3: $C(2,2) \times 2!$
 $[cite_start]= 1 \times 2 = 2$ Multiplicando as combinações desta partição: $56 \times 6 \times 10 \times 6 \times 1 \times 2 = 40.320$ combinações. Somando as restantes partições de distribuição válidas, o espaço amostral total situa-se na ordem de 10^5 a 10^6 soluções possíveis.

b) A complexidade assintótica da busca exaustiva (força bruta) no pior cenário é expressa por $O(m^n \times n!)$. O fator exponencial m^n traduz todas as atribuições possíveis de n pedidos aos m estafetas disponíveis, enquanto o fator fatorial $n!$ modela a explosão combinatória associada à ordenação das paragens de recolha e entrega ao longo das rotas.

c) Comparação numérica do crescimento assintótico:

Tamanho da Entrada (n)	Crescimento Polinomial $O(n^2)$	Crescimento Fatorial $O(n!)$
$n = 8$	64	40.320
$n = 15$	225	$\approx 1,3 \times 10^{12}$
$n = 20$	400	$\approx 2,4 \times 10^{18}$

Estes resultados evidenciam uma divergência assintótica radical: enquanto a função polinomial apresenta um crescimento controlado, a função fatorial expande-se de forma superexponencial.

d) Com base nestes dados, um processador moderno capaz de executar 10^9 operações por segundo demoraria cerca de $2,4 \times 10^9$ segundos (aproximadamente 76 anos) para avaliar exaustivamente uma instância moderada de $n=20$ pedidos. Logo, a adoção

de algoritmos de força bruta torna-se absolutamente inviável para o sistema da FastBite, cuja restrição operacional estrita impõe um teto de 2 segundos de processamento.

QUESTÃO 3: Algoritmo Guloso e Análise de Contraexemplo

a) Aplicando a fórmula da distância Manhattan $|x_1 - x_2| + |y_1 - y_2|$ passo a passo no cenário fornecido:

Pedido P1 (Restaurante em (0,2)): Distâncias iniciais: $E1 \rightarrow |0-1|+|2-1| = 2\$$; $E2 \rightarrow |0-5|+|2-3| = 6\$$; $E3 \rightarrow |0-7|+|2-6| = 11\$$. O estafeta **E1** é o mais próximo e assume P1 (Capacidade restante: 1).

Pedido P2 (Restaurante em (1,1)): Distâncias: $E1 \rightarrow |1-1|+|1-1| = 0\$$; $E2 \rightarrow |1-5|+|1-3| = 4\$$; $E3 \rightarrow |1-7|+|1-6| = 11\$$. O estafeta **E1** assume P2 por estar na mesma coordenada (E1 esgota a sua capacidade de 2 pedidos).

Pedido P3 (Restaurante em (4,4)): Com E1 indisponível, avalia-se: $E2 \rightarrow |4-5|+|4-3| = 2\$$; $E3 \rightarrow |4-7|+|4-6| = 5\$$. O estafeta **E2** recebe P3. O algoritmo repete sucessivamente este critério de proximidade imediata para P4 e P5, estruturando depois as rotas internas seguindo a heurística do vizinho mais próximo a partir da posição corrente de cada veículo.

b) Este algoritmo é classificado como guloso (*greedy*) porque adota uma propriedade de escolha local míope ("atribuir imediatamente o recurso mais próximo no instante atual"), assumindo que uma sucessão de decisões ótimas locais conduzirá necessariamente a um resultado global perfeito, sem reconsiderar decisões anteriores ou avaliar consequências futuras.

c) **Contraexemplo:** Ao alocar o pedido P3 ao estafeta E2 por critérios gulosos de proximidade (distância de 2), o algoritmo impede que E2 fique livre para recolher os pedidos P4 (distância de 3) e P5 (distância de 4) independentemente dos subproblemas. No particionamento geográfico da FastBite, esta premissa é violada, pois as decisões tomadas numa zona afetam as adjacentes sempre que ocorrem pedidos cujos restaurantes e clientes estão em quadrantes distintos (*cross-zone*).

b) Dividindo o plano bidimensional cartesiano pelo ponto médio das coordenadas do cenário (4, 3.5), os pedidos P1 (restaurante em (0,2)) e P2 (restaurante em (1,1)) são mapeados na zona Sudoeste (SW). Os pedidos P3 (4,4) e P5 (5,5) posicionam-se na zona Nordeste (NE), e o pedido P4 (restaurante em (6,1)) fica retido na zona Sudeste (SE). No que toca aos recursos, E1 (1,1) pertence à zona SW, E3 (7,6) à zona NE, e E2 (5,3) localiza-se na fronteira.

c) As limitações manifestam-se agressivamente nas fronteiras: o pedido P4 possui o seu restaurante na zona SE e o cliente na zona Noroeste (NW) em (3,6). O particionamento geográfico estrito força a resolução isolada em SE, ignorando de forma cega que o estafeta E1 (da zona SW) poderia finalizar a entrega de forma mais eficiente. Da mesma forma,

isolar o estafeta E2 numa única zona remove a sua capacidade natural de mitigar a procura em múltiplos quadrantes limítrofes.

d) Face ao algoritmo guloso da Questão 3, a Divisão e Conquista garante uma redução drástica no espaço de busca computacional dentro de cada quadrante individual. Contudo, sacrifica severamente a qualidade da solução global devido à perda de visão sistémica sobre as fronteiras. O algoritmo guloso, embora localmente míope, adapta-se de forma mais fluida a distribuições geográficas contínuas por não possuir barreiras geométricas rígidas artificiais.

QUESTÃO 6: Heurísticas e Engenharia de Soluções Reais

a) No contexto de otimização, uma heurística é uma estratégia algorítmica que orienta a busca por soluções satisfatórias de forma rápida, prescindindo de garantias matemáticas de otimalidade global. Difere de um algoritmo exato (que assegura matematicamente o ótimo global a um custo computacional tipicamente exponencial) e de um algoritmo de aproximação (que oferece uma garantia formal de que o resultado gerado estará contido num fator limite k de erro em relação ao ótimo absoluto).

b) Justificação técnica das quatro etapas da indústria real:

Particionamento Geográfico: Restringe o escopo prático do algoritmo, reduzindo a entrada volumétrica de 80.000 pedidos diários para subconjuntos manejáveis de poucas centenas por zona regional.

Heurística Gulosa Inicial: Computa uma primeira solução válida e estruturada a um custo polinomial baixo $O(n \log n)$, assegurando estabilidade operacional imediata.

Refinamento Local (ex: 2-opt): Realiza permutas e inversões cirúrgicas de rotas sobre a solução inicial, elevando a qualidade do resultado final sem explorar o espaço combinatório completo.

Limite de Tempo Estrito: Interrompe os ciclos de refinamento local após um intervalo predefinido (ex: 1,5s), salvaguardando a previsibilidade em tempo real do sistema produtivo.

c) O cálculo exato justifica o custo computacional quando o número de paragens atribuídas a um estafeta individual é muito reduzido ($k \leq 3$). Com $3! = 6$ permutações totais, a força bruta encontra o ótimo absoluto em microssegundos. Também se aplica a cenários de planeamento logístico *offline* ou pré-roteamento noturno para o dia seguinte, onde o tempo de execução não é uma variável crítica.

d) Em engenharia de sistemas de larga escala, o "bom o suficiente" representa a decisão técnica mais correta quando o custo marginal financeiro e computacional para extrair os últimos percentuais de otimização exata supera os benefícios práticos do negócio. Num ambiente concorrente de alta frequência, a capacidade de emitir milhares de decisões rápidas e 95% ótimas é manifestamente superior a colapsar o sistema produtivo à espera

de soluções matemáticas 100% perfeitas.

FICHEIRO 2: ESTRUTURAS DE DADOS LINEARES E COMPLEXIDADE

QUESTÃO 1: Undo/Redo e Escolha de Estrutura

a) A **Pilha** é uma estrutura de dados linear regida pela política **LIFO** (*Last In, First Out*), na qual o último elemento inserido é obrigatoriamente o primeiro a ser removido; as suas operações básicas são o **push** (inserção) e o **pop** (remoção), executados exclusivamente na extremidade ativa denominada topo. O **Vetor** (ou *array*) configura um arranjo linear de elementos alocados de forma contígua na memória, suportando inserções, remoções e acesso direto indexado a qualquer posição.

b) Análise do custo assintótico utilizando a notação Big-O:

Inserção: No início: Vetor $O(n)$ / Pilha $O(1)$ (topo). No fim: Vetor $O(1)$ amortizado / Pilha $O(1)$. No meio: Vetor $O(n)$ / Pilha: operação não permitida de forma direta.

Remoção: No início: Vetor $O(n)$ / Pilha $O(1)$ (topo). No fim: Vetor $O(1)$ / Pilha $O(1)$. No meio: Vetor $O(n)$ / Pilha: operação não permitida de forma direta.

c) A pilha constitui a estrutura ideal para sistemas de *undo/redo*, dado que este requisito operacional exige intrinsecamente a recuperação das ações do utilizador na ordem inversa à da sua execução original. Como o **push** e o **pop** operam num custo constante estável de $O(1)$, a fluidez do sistema é preservada independentemente da volumetria acumulada de dados.

d) Se a funcionalidade fosse suportada por um vetor e o programador optasse por efetuar as remoções no início da estrutura (índice 0) para reverter as operações, cada clique em "Desfazer" exigiria o deslocamento em memória de todos os restantes elementos subsequentes. Num cenário com milhares de ações registadas, este custo linear $O(n)$ degradaria a performance da aplicação, gerando bloqueios e latência perceptível no ecrã.

QUESTÃO 2: Filas em Sistemas de Mensageria

a) A **Fila** convencional baseia-se na política **FIFO** (*First In, First Out*), onde o primeiro elemento introduzido é necessariamente o primeiro a ser removido, disponibilizando as operações de **enqueue** (enfileirar no fim) e **dequeue** (desenfileirar no início). O **Deque** (*double-ended queue*) atua como uma extensão linear flexível que aceita operações de inserção e remoção de alta performance em ambas as extremidades (tanto no início como no fim).

b) Numa implementação otimizada (como listas encadeadas com ponteiros para as extremidades ou vetores circulares), os custos assintóticos da fila para enfileirar e desenfileirar são de tempo constante $O(1)$. No deque, as operações de inserção e

remoção em qualquer um dos pólos (frontal ou traseiro) mantêm-se em $\$O(1)\$$. O acesso direto aos elementos das extremidades é de $\$O(1)\$$ em ambos os cenários.

c) Um exemplo real reside nos servidores de jogos online ou nos sistemas de streaming de eventos. O fluxo geral de pacotes é enfileirado na retaguarda da estrutura em $\$O(1)\$$, contudo, quando surge um comando crítico do administrador ou uma notação de desconexão urgente de um utilizador, o sistema beneficia do uso do deque para injetar essa mensagem prioritária diretamente na frente da estrutura ($\$O(1)\$$), furando a fila de espera comum.

d) A substituição destas estruturas por um vetor linear simples comprometeria o desempenho, pois a operação de remoção no início (equivalente ao **dequeue**) importaria um custo de $\$O(n)\$$. O processador seria forçado a mover todos os elementos restantes uma posição para a esquerda para preencher o espaço vago na memória contígua, gerando um estrangulamento catastrófico inviável para altos volumes de dados.

QUESTÃO 3: Histórico de Navegação e Listas Encadeadas

a) O comportamento esperado de um histórico exige três ações fundamentais : **voltar** (recuar para a URL anterior), **avancar** (progredir para a URL seguinte) e **nova_visita** (inserir uma nova página a partir da posição atual, obrigando ao descarte imediato de todo o "histórico futuro" remanescente). Isto requer operações eficientes de navegação bidirecional e eliminação/inserção de nós intermédios.

b) Comparação de custos assintóticos:

Navegação (Voltar/Avançar): Vetor Dinâmico: $\$O(1)\$$ por indexação / Lista Duplamente Encadeada: $\$O(1)\$$ através da movimentação dos ponteiros **prev** e **next**.

Truncar Histórico Futuro / Inserção Intermédia: Vetor Dinâmico: $\$O(n)\$$, devido ao custo associado à eliminação em bloco e realocação de memória. Lista Duplamente Encadeada: $\$O(1)\$$, pois basta reajustar os ponteiros de endereço do nó corrente para apontarem para o novo link, isolando o subconjunto futuro instantaneamente.

c) A **lista duplamente encadeada** assume-se como a estrutura ideal para este cenário. O acesso aleatório por índice fornecido pelo vetor não traz vantagens reais, dado que a navegação do utilizador no histórico é eminentemente sequencial. O benefício fulcral da lista reside na sua capacidade de descartar e acoplar novos fluxos de páginas em tempo constante $\$O(1)\$$.

d) Se o browser utilizasse um vetor dinâmico, considere um cenário onde o utilizador navega por 500 páginas, retrocede 200 posições e clica num novo endereço. Para registar esta nova visita, o vetor seria forçado a reestruturar os seus índices ou deslocar elementos para eliminar as 200 páginas órfãs do histórico futuro, operando em tempo linear $\$O(n)\$$. Em dispositivos com memória contígua limitada (como smartphones), esta operação causaria picos de latência e consumo ineficiente de recursos.

QUESTÃO 4: Criptografia e os Limites da Força Bruta

a) A **Classe P** delimita os problemas computacionais que admitem soluções determinísticas eficientes com tempo de execução polinomial. A **Classe NP** abrange problemas em que a validação de uma solução candidata pode ser efetuada eficientemente em tempo polinomial, embora a sua resolução direta seja complexa. No criptosistema RSA, multiplicar dois números primos gigantescos para gerar uma chave pública é uma tarefa em P (rápida), e confirmar se dois fatores dividem o número gerado é uma verificação em NP (eficiente). Contudo, o ato inverso de descobrir esses fatores a partir do composto (fatoração) carece de algoritmos polinomiais conhecidos.

b) Problemas cujo espaço de busca cresce de forma exponencial impõem barreiras intransponíveis à força bruta porque o número de combinações possíveis duplica ou expande-se geometricamente a cada incremento elementar na entrada (como adicionar bits a uma chave). A pesquisa exaustiva torna-se computacionalmente inviável, dado que o tempo necessário para processar todas as ramificações de busca ultrapassa rapidamente limites físicos, como a idade estimada do universo.

c) Um exemplo prático e central é o **Problema da Fatoração de Inteiros Grandes** que confere robustez matemática à criptografia assimétrica RSA, ou a quebra de chaves do padrão simétrico **AES-256** por intermédio da enumeração exaustiva de chaves.

d) Para contornar estas restrições, utilizam-se estratégias alternativas como heurísticas de fatoração avançadas (ex: crivo quadrático) ou algoritmos probabilísticos de teste de primalidade (ex: teste de Miller-Rabin). Estas abordagens reduzem drasticamente o tempo de computação, mas sacrificam a determinação absoluta: algoritmos probabilísticos operam sob uma margem de erro controlada e não asseguram a obtenção da solução ótima ou exata em 100% das execuções.

QUESTÃO 5: Escalonamento de Tarefas em Nuvem

a) Problemas na classe P possuem algoritmos determinísticos de resolução polinomial; em NP, a verificação da solução é polinomial; e os problemas **NP-Completo** representam os desafios mais complexos contidos em NP, interligados por reduções polinomiais recíprocas. O problema de escalonamento de tarefas (correlato ao clássico *bin packing*) está integrado na categoria dos problemas **NP-Completo**.

b) À medida que o volume de tarefas n escalabilidade em direção a patamares de pico, o mapeamento exaustivo de todas as combinações de alocação de cargas de trabalho perante os servidores físicos m expande-se de modo combinatório. A busca pela solução ótima absoluta gera uma árvore de exploração cujo tempo de processamento explode de forma exponencial, inviabilizando qualquer resposta em tempo real.

c) O *bin packing* consiste em acomodar itens de dimensões variáveis (tarefas computacionais) no menor número possível de recipientes com restrições de volume (capacidade de CPU/RAM dos servidores). O fator que despoleta a complexidade exponencial é a interdependência das decisões: cada tarefa alocada restringe e altera a capacidade residual da máquina para as tarefas seguintes, gerando um espaço combinatório de busca massivo.

d) Uma estratégia aproximada eficiente é a aplicação do algoritmo guloso **First Fit Decreasing (FFD)**. Esta heurística ordena as tarefas de forma decrescente de tamanho e insere cada uma no primeiro servidor físico que contiver capacidade disponível. Garante-se uma execução de excelente performance polinomial em tempo $O(n \log n)$, com o benefício técnico formal de assegurar um resultado aproximado contido no limite estrito de $\frac{11}{9} \cdot \text{OPT} + \frac{6}{9}$ caixas. Sacrifica-se a otimalidade perfeita, mas viabiliza-se a operação contínua da nuvem.

QUESTÃO 6: Coloração de Grafos em Compiladores

a) A classe P abrange soluções polinomiais; NP requer verificações polinomiais; e a classe NP-Completo reúne os problemas mais difíceis da categoria NP, redutíveis entre si. O problema de mapear a quantidade mínima de cores necessárias para colorir um grafo de tal forma que dois vértices adjacentes nunca partilhem a mesma cor (número cromático) é formalmente classificado como **NP-Completo**.

b) Encontrar a coloração ótima torna-se computacionalmente inviável em grafos de grande escala porque o espaço de soluções candidatas expande-se de forma exponencial com o crescimento do número de vértices. Para estruturas complexas, o tempo de processamento exigido para garantir a solução mínima causaria o bloqueio e a paragem do compilador durante a geração do binário.

c) A alocação de registradores em compiladores como GCC e LLVM usa esta modelação: cada variável do código é mapeada como um vértice num **grafo de interferência**. Uma aresta é traçada para ligar duas variáveis caso ambas estejam ativas em simultâneo no fluxo de execução, competindo pelo mesmo recurso físico. As cores do grafo representam a quantidade finita de registradores físicos rápidos que o processador possui.

d) A indústria contorna esta limitação implementando o **Algoritmo Guloso de Coloração de Chaitin**. Esta estratégia ordena os vértices de acordo com o seu grau de interferência e atribui a cada um, sequencialmente, a menor cor válida disponível no momento, operando sob um custo polinomial linear altamente eficiente de $O(V+E)$. Sacrifica-se a otimalidade cromática (o compilador pode falhar em usar o mínimo teórico de cores, sendo forçado a efetuar o *spill* de variáveis para a memória RAM mais lenta). Contudo, este compromisso é aceitável, pois assegura que softwares de grande escala sejam compilados em tempos viáveis e céleres.

) e P5 (distância de 2). Numa alocação alternativa não gulosa, se P3 fosse atribuído ao estafeta E3 (distância de 5), o somatório total das rotas do sistema seria significativamente inferior, provando que a estratégia gulosa fecha janelas de otimização global.

d) A complexidade de tempo do algoritmo divide-se em duas etapas: a fase de atribuição requer avaliar m estafetas para cada um dos n pedidos, resultando em $O(n \times m)$. A fase de roteamento individual consome $O(k^2)$ por estafeta (vizinho mais próximo), totalizando $O(n^2 / m)$ para todo o sistema. A complexidade final combinada é expressa por $O(n \times m + n^2 / m)$.

QUESTÃO 4: Programação Dinâmica no Roteamento Individual

a) Para a aplicação de Programação Dinâmica (PD), são exigidas duas condições: **subestrutura ótima** (a solução ótima do problema contém em si as soluções ótimas dos seus subproblemas) e **sobreposição de subproblemas** (os mesmos subproblemas são recalculados repetidamente na busca exaustiva). O roteamento individual de um estafeta com k paragens cumpre estes requisitos, visto que um caminho ótimo de A até Z que intersepte um subconjunto de paragens S conterá, obrigatoriamente, a sub-rotina ótima para qualquer subconjunto de S , e o custo de atingir um determinado vértice através de diferentes permutações do mesmo subconjunto é repetido na árvore de busca.

b) O subproblema neste contexto modela o algoritmo de Held-Karp para o TSP. O estado $dp[S][v]$ armazena (memoiza) o custo mínimo para visitar todas as paragens contidas no subconjunto S , terminando especificamente na paragem v . A equação de transição de estados define-se como: $dp[S][v] = \min_{u \in S \setminus \{v\}} \{ dp[S \setminus \{v\}][u] + \text{dist}(u,v) \}$, para todo o $u \in S \setminus \{v\}$.

c) A complexidade de tempo desta abordagem é de $O(2^k \times k^2)$, dado que existem 2^k subconjuntos combinatórios possíveis de paragens, k destinos finais potenciais por estado e k transições internas a computar. A complexidade de espaço é demarcada por $O(2^k \times k)$, necessária para manter os estados na tabela de memoização. O parâmetro k dita o custo devido ao crescimento exponencial do fator 2^k .

d) Para a configuração padrão da FastBite ($k \leq 3$ pedidos por estafeta), a PD processa apenas $O(2^3 \times 3^2) = O(72)$ operações, sendo computacionalmente trivial e instantânea. Avaliando limites: para $k=10$, consome $\approx 10^5$ operações (rápido); para $k=20$, atinge $\approx 4 \times 10^8$ operações (limiar dos 2 segundos); e para $k=25$, exige $\approx 2 \times 10^{10}$ operações, tornando-se impraticável. Conclusão: com o limite operacional de $k \leq 3$, a PD é perfeitamente viável para o roteamento individual; o verdadeiro estrangulamento do sistema reside na fase prévia de atribuição coletiva.

QUESTÃO 5: Divisão e Conquista por Particionamento Geográfico

a) O princípio de Divisão e Conquista consiste em cindir um problema complexo em subproblemas menores e estritamente independentes, resolvê-los isoladamente e combinar os seus resultados para formular a solução ótima global. A condição basilar é a